NASA Space Apps Challenge 2026 — Project Specification: Autonomous Lunar Landing Navigator & 3D Mission Simulator (NASA-HACK)

Event: 2026 NASA Space Apps Challenge | **Track:** Lunar Landing Site Navigator / Space Mission Design Game | Repository: https://github.com/Marwane-Haddane/nasa-hack | Team Size: 3 Engineering Students | Target Vehicle: Dropper X / Nova-C Autonomous Lunar Drone Hopper
|---|---|---|

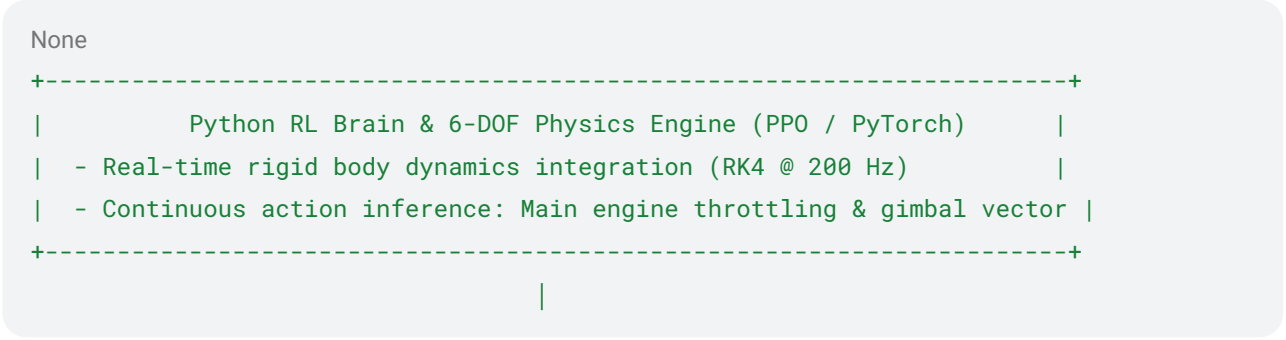
1. Executive Summary & Core Value Proposition

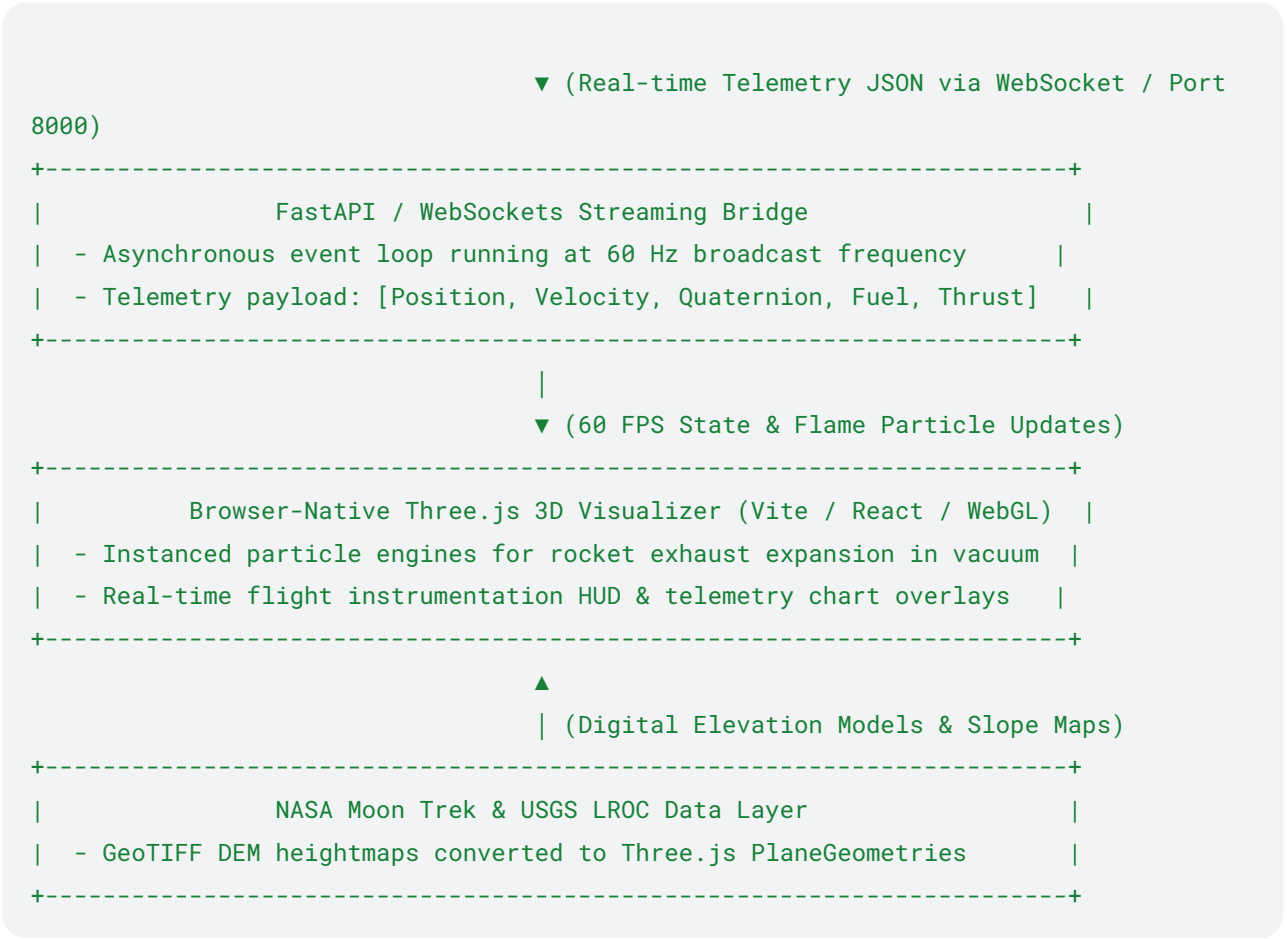
Autonomous precision soft landing on the lunar surface remains one of the most critical challenges in modern aerospace engineering. NASA mission planning requires balancing landing terrain hazards—such as boulder fields, steep slopes, and shadow regions—with strict mass and propellant limits. Existing mission design platforms present a major trade-off: they are either static 2D terrain maps lacking dynamic guidance feedback, or high-overhead desktop software suites (e.g., GMAT, STK) that are inaccessible for rapid web-based visualization, real-time interactive prototyping, and educational outreach.

The Solution: NASA-HACK introduces a hybrid, browser-native autonomous lunar landing navigation platform. By combining a 6-Degree-of-Freedom (6-DOF) Deep Reinforcement Learning (Proximal Policy Optimization - PPO) Guidance, Navigation, and Control (GNC) engine with a high-performance Three.js visualizer running over WebGL, the system delivers real-time flight path optimization, terrain hazard avoidance, and full 60 FPS telemetry streaming directly within standard web browsers.

2. System Architecture & Connection Flow

The system uses a decoupled microservice architecture. A high-performance Python backend runs the continuous-space 6-DOF physics engine alongside PyTorch PPO inference models. Low-latency telemetry state vectors are broadcast across an asynchronous WebSocket streaming bridge to the client interface, which blends telemetry with digital elevation models (DEMs).





3. Subsystem Breakdown & Image Annotations

3.1 6-DOF Reinforcement Learning Guidance & Telemetry Engine

The core GNC system utilizes a continuous Proximal Policy Optimization (PPO) agent trained within a custom 6-DOF rigid-body lunar simulator (inspired by the open-source rocket landing simulator by aryamehr2k).

The state space $\mathbf{s} \in \mathbb{R}^{14}$ includes:
$$\mathbf{s} = [x, y, z, \dot{x}, \dot{y}, \dot{z}, q_0, q_1, q_2, q_3, \omega_x, \omega_y, \omega_z, m_{\text{fuel}}]$$

The continuous control action $\mathbf{a} \in \mathbb{R}^{3}$ governs main engine thrust magnitude $T_{\text{cmd}} \in [0.1, 1.0]$, pitch gimbal angle $\delta_p \in [-20^\circ, +20^\circ]$, and yaw gimbal angle $\delta_y \in [-20^\circ, +20^\circ]$. Differential Cold Gas Thrusters (RCS) supply attitude correction moments.

Hover constraints enforce a velocity vector alignment within a tight descent cone ($\theta_{\text{max}} = 15^\circ$), and touchdown parameters mandate a touchdown vertical velocity

$v_z \le 2.0 \text{ m/s}$, horizontal velocity $v_{xy} \le 0.5 \text{ m/s}$, and pitch/roll inclination off-axis $\theta_{\text{tilt}} \le 5.0^\circ$.

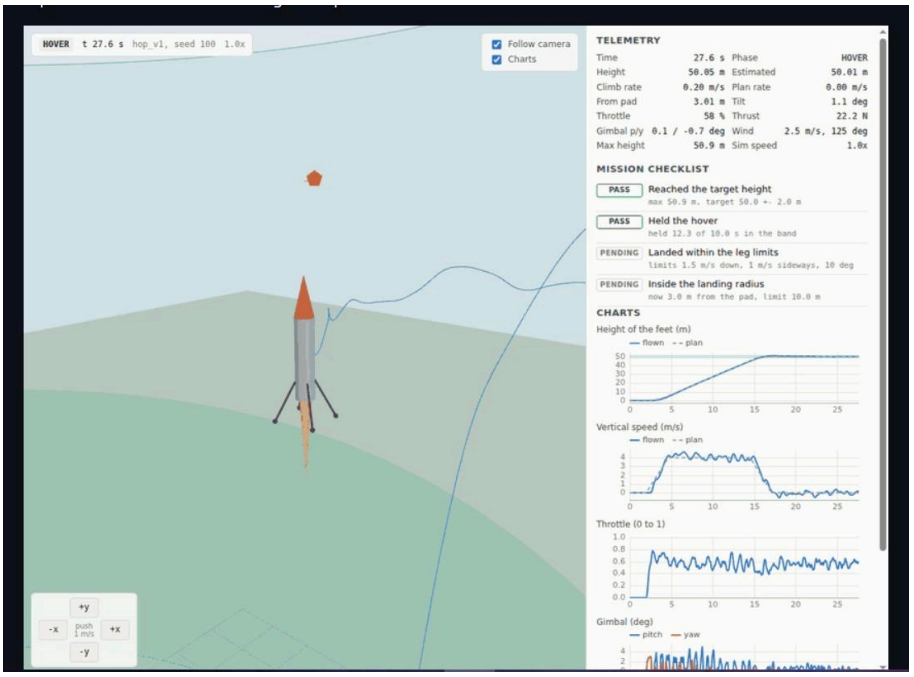


Figure 1: Live 6-DOF Flight Telemetry HUD displaying altitude-versus-downrange trajectory curves, attitude roll/pitch/yaw tracking, fuel consumption metrics, and real-time mission execution checklists.

3.2 Browser-Native 3D Visualization & Flight HUD

The rendering subsystem is built with Three.js via Vite and React (incorporating elements adapted from [OsamaZerkawi's Rocket Simulation]).

- **Rendering Pipeline:** Utilizes WebGL2 with custom GLSL shaders for lunar dust agitation during engine burn proximity. High-resolution shadow maps dynamically render sunlight incidence angles at the lunar south pole (e.g., Shackleton Crater).
- **Lighting Engine:** Single distant directional light source simulating unfiltered solar radiation (1361 W/m^2) paired with faint ambient lunar regolith bounce lighting.
- **Particle Systems:** GPU-accelerated particle system modeling engine exhaust plume expansion in a vacuum environment (governed by the polar expansion angle formula).
- **Interactive Parameters:** Live controls allow users to modify initial drop altitude, horizontal wind disturbance injection (testing GNC robustness), engine gimbal limits, and target landing coordinates on the fly.



Figure 2: Interactive 3D scene displaying the Dropper X lunar vehicle during powered descent, real-time propulsive vector arrows, launchpad ground markers, and dynamic vehicle telemetry readouts.

3.3 End-to-End Multi-Phase Mission Workflow

The automated flight trajectory follows a structured five-phase mission progression sequence:

1. **Powered Ascent / Initial Deorbit Burn:** Deorbit impulse establishes an elliptical transfer orbit down to a perilune altitude of 15 km.
2. **Ballistic Drift Phase:** Unpowered coasting phase optimizing propellant reserves while the onboard Lidar scans crater rim topographies.
3. **Autonomous Divert Phase:** The PPO neural policy evaluates real-time hazard maps generated from DEM data, executing lateral thruster burns to adjust the targeted touchdown site away from detected boulders.
4. **Suicide Burn (Terminal Deceleration):** The main variable-thrust engine ignites at maximum thrust to decelerate the lander, smoothly throttling down along a parabolic braking curve.
5. **Touchdown & Debrief:** Engine cutoff triggers upon touchdown sensor activation, transitioning the system into an automated post-flight telemetry analysis phase.



Figure 3: Multi-phase mission workflow screen displaying stage-by-stage vehicle state transitions, trajectory phase indicators, orbital-to-surface descent track, and post-flight telemetry summaries.

4. Mathematical & Physical Environment Models

4.1 Lunar Physical Constants

- **Gravitational Acceleration:** $g_{\text{lunar}} = 1.62 \text{ m/s}^2$ directed along the negative Z-axis ($\mathbf{g} = [0, 0, -1.62]^T$).
- **Atmospheric Density:** $\rho_{\text{lunar}} = 0.0 \text{ kg/m}^3$ (Vacuum environment; aerodynamic drag force $F_D = 0$ and lift force $F_L = 0$).
- **Lander Mass Properties:** Dry mass $m_{\text{dry}} = 450 \text{ kg}$, maximum propellant mass $m_{\text{fuel},0} = 650 \text{ kg}$, variable mass $m(t) = m_{\text{dry}} + m_{\text{fuel}}(t)$.

4.2 PPO Reward Function Formulation

The multi-objective reward function R_t maximizes control precision while minimizing fuel usage:

$$R_t = R_{\text{distance}} + R_{\text{velocity}} + R_{\text{attitude}} + R_{\text{fuel}} + R_{\text{touchdown}}$$

Where:

- **Distance Penalty:** $R_{\text{distance}} = -\alpha \cdot |\mathbf{r}_{\text{lander}} - \mathbf{r}_{\text{target}}|_2$
- **Velocity Penalty:** $R_{\text{velocity}} = -\beta \cdot |\mathbf{v}_{\text{lander}} - \mathbf{v}_{\text{target}}|_2$

- **Attitude Alignment:** $R_{\text{attitude}} = +\gamma \cdot (\mathbf{z}_{\text{body}} \cdot \mathbf{z}_{\text{world}})$ (encourages staying upright)
- **Fuel Economy:** $R_{\text{fuel}} = -\delta \cdot \left(\frac{T_{\text{cmd}}}{T_{\text{max}}} \right)$
- **Touchdown Event:**
  $$R_{\text{touchdown}} = \begin{cases} +500 & \text{if } v_z \le 2.0 \text{ m/s}, v_{xy} \le 0.5 \text{ m/s}, \theta \le 5^\circ \\ -300 & \text{if structural limits exceeded (crash landing)} \end{cases}$$

5. 3D Assets & Hardware Integration

The landing platform models the geometry and mass properties of autonomous lunar hoppers (e.g., Dropper X / Intuitive Machines Nova-C). CAD structures were adapted from open-source aerospace models, optimized for real-time WebGL rendering via GLTF/GLB compression pipeline:

- **Nova-C / IM-1 Lander Geometry:** Printables Model 821185
- **Lunar Lander Assembly:** GrabCAD Structural Asset
- **Hardware & Avionics Register:** Detailed subsystem components (IMU, Lidar Altimeter, Main Throttle Valves, RCS Thruster Blocks) are documented in the project's Component Reference Register.

6. Repository Structure & File Hierarchy

The project is structured as a unified monorepo located at https://github.com/Marwane-Haddane/nasa-hack:

```
None
nasa-hack/
├── README.md
├── docker-compose.yml
├── backend/
│   ├── main.py                  # FastAPI WebSocket entrypoint
│   ├── requirements.txt         # PyTorch, FastAPI, NumPy, SciPy
│   ├── physics/
│   │   ├── lunar_environment.py  # 6-DOF rigid body equations of motion
│   │   └── dem_parser.py         # GeoTIFF/LROC data parser
│   └── gnc/
│       ├── ppo_agent.py          # PyTorch PPO Policy Network architecture
│       ├── evaluate.py           # Real-time GNC inference engine
│       └── weights/
│           └── lunar_ppo_v4.pt   # Pre-trained neural network weights
```

```
├── frontend/
│   ├── package.json              # React, Three.js, Vite dependencies
│   ├── vite.config.js
│   ├── src/
│   │   ├── App.jsx               # Main dashboard UI component
│   │   ├── components/
│   │   │   ├── Canvas3D.jsx      # Three.js viewport render container
│   │   │   ├── TelemetryHUD.jsx  # Real-time SVG flight overlay & charts
│   │   │   └── Controls.jsx      # Environment & parameter control panel
│   │   ├── graphics/
│   │   │   ├── LanderModel.jsx   # GLTF vehicle loader & exhaust shaders
│   │   │   └── TerrainMesh.jsx   # Dynamic elevation map generator
│   │   └── services/
│   │       └── websocket.js      # 60Hz telemetry stream handler
├── assets/
│   ├── models/
│   │   └── dropper_x.glb         # High-poly GLTF Lunar Lander model
│   └── dem/
│       └── shackleton_south_pole.tif
└── docs/
    ├── architecture.png
    └── mathematical_model.pdf
```

7. 3-Member Team Division of Labor

Member	Domain & Role	Primary Responsibilities
Member 1	AI, GNC & Lunar Dynamics Lead	Design and implementation of the 6-DOF continuous rigid-body physics simulator; PPO neural network training in PyTorch; tuning landing reward functions and hover stability constraints.
Member 2	Frontend, Three.js & UI/UX Lead	Development of the WebGL rendering pipeline in Three.js/React; implementation of custom particle shaders for vacuum engine plumes; creation

Member	Domain & Role	Primary Responsibilities
		of the real-time telemetry HUD overlay.
Member 3	Systems Integration, NASA Datasets & Communication Lead	Implementation of the FastAPI WebSocket telemetry streaming bridge; ingestion and processing of NASA Moon Trek DEM heightmaps; repository maintenance and documentation.

8. Step-by-Step Implementation Plan & 3-Person Team Execution Roadmap (Winning Strategy for NASA Space Apps 2026)

To deliver a competitive, award-winning submission for the NASA Space Apps Challenge 2026, the team follows a structured 48-hour engineering roadmap. This plan converts static baseline open-source tools into an integrated, real-time lunar landing simulation suite.

8.1 Target Monorepo & Git Workflow

All code, models, terrain data, and UI assets are integrated into a single monorepo hosted at https://github.com/Marwane-Haddane/nasa-hack.

- **main:** Production-ready stable release branch for final NASA judges demonstration and deployment.
- **dev-backend-gnc:** 6-DOF physics engine, PPO PyTorch agent training, and flight dynamics simulation.
- **dev-frontend-3d:** Three.js scene, WebGL particle system, React UI, and flight HUD overlays.
- **dev-integration:** FastAPI WebSocket telemetry streaming bridge and NASA Moon Trek DEM terrain ingestion.

8.2 Repositories to Clone & Mandatory Modifications to Win

The platform leverages open-source foundations but adapts them significantly for true lunar vacuum flight physics and real-time web rendering:

- **Primary Backend Engine:** aryamehr2k/rocket-landing-rl-simulator
- **Primary Frontend 3D Visualizer:** OsamaZerkawi/Rocket_Simulation
- **3D CAD Vehicle Assets:** Printables Model 821185 (Nova-C / IM-1) & GrabCAD Lunar Lander Assembly

Why Unmodified Clones Will Lose & What Must Be Modified:

- **Physics Modification:** Change Earth gravity ($g = -9.81 \text{ m/s}^2$) to Lunar gravity ($g = -1.62 \text{ m/s}^2$). Zero out atmospheric drag ($C_d = 0, \rho = 0$) to enforce true vacuum physics, requiring attitude control to rely strictly on TVC gimbal vectoring and cold-gas RCS pulses.
- **Vehicle Modification:** Replace generic Earth rocket cylinder geometries with the Dropper X / Nova-C 3D GLTF lander model, matching true dry mass (450 kg), propellant mass (650 kg), and 4-point landing gear footpad collision bounding boxes.
- **Environment Modification:** Strip default green vegetation/Earth terrain and ingest NASA Moon Trek (https://trek.nasa.gov/moon/) South Pole Shackleton Crater rim elevation data with low-angle directional solar illumination.
- **RL Guidance Modification:** Retrain the PPO neural network policy using a shaped reward function engineered specifically for suicide-burn timing, fuel conservation, and precision touchdown limits ($v_z \le 1.5 \text{ m/s}, \theta_{\text{tilt}} \le 5^\circ$).
- **System Bridge:** Connect the Python 6-DOF physics backend to the browser Three.js visualizer via an asynchronous 60 Hz bidirectional WebSocket streaming pipeline.
- **User Experience:** Add a "Manual Pilot vs. AI Autopilot" interactive mode toggle so NASA judges can attempt landing manually under lunar inertia before reviewing the AI autonomous flight.

8.3 Granular Task Repartition for the 3-Person Team

Member 1: AI, GNC & Lunar Flight Dynamics Lead

- **Task 1.1:** Clone the aryamehr2k repository into the `backend/` directory.
- **Task 1.2:** Refactor physics integration algorithms to enforce $g_{\text{lunar}} = 1.62 \text{ m/s}^2$ and vacuum conditions ($\rho = 0$).
- **Task 1.3:** Define the 15-variable state vector $\mathbf{s}$ and 3-variable continuous action space $\mathbf{a}$ (main thrust throttle, pitch gimbal, yaw gimbal).
- **Task 1.4:** Implement shaped reward functions governing touchdown velocity constraints, attitude alignment, and propellant conservation.
- **Task 1.5:** Execute PyTorch PPO neural network training for 500,000+ timesteps and export trained weights to `lunar_ppo_v4.pt`.
- **Task 1.6:** Commit and push backend code to https://github.com/Marwane-Haddane/nasa-hack.

Member 2: Frontend, Three.js 3D Graphics & UI/UX Lead

- **Task 2.1:** Clone OsamaZerkawi/Rocket_Simulation into the `frontend/` directory.
- **Task 2.2:** Initialize Vite/React application, remove Earth asset dependencies, and construct 3D deep space skybox and directional solar lighting.

- **Task 2.3:** Load Dropper X / Nova-C GLTF model with dynamic engine nozzle gimbaling and vacuum exhaust plume particle shaders.
- **Task 2.4:** Develop real-time telemetry HUD overlay displaying altitude, descent velocity, propellant percentage, gimbal angle, and trajectory curves.
- **Task 2.5:** Build interactive Mission Checklist panel with real-time pass/fail flight status indicators.
- **Task 2.6:** Commit and push frontend code to https://github.com/Marwane-Haddane/nasa-hack.

Member 3: Systems Integration, WebSocket Bridge & NASA Lead

- **Task 3.1:** Initialize team monorepo, Docker Compose orchestration configuration, and GitHub Actions CI/CD pipelines at https://github.com/Marwane-Haddane/nasa-hack.
- **Task 3.2:** Construct FastAPI WebSocket server (`main.py`) to stream state telemetry JSON payloads at 60 Hz.
- **Task 3.3:** Parse NASA Moon Trek GeoTIFF elevation heightmaps and convert DEM datasets into Three.js `PlaneGeometry` terrain meshes.
- **Task 3.4:** Implement Manual vs. AI flight control toggle along with color-coded terrain slope hazard visualization (< 5° safe green, > 15° hazard red).
- **Task 3.5:** Record 30-second high-resolution demonstration video, author project documentation, and finalize NASA Space Apps Challenge portal submission.
- **Task 3.6:** Verify unified single-command system execution (`npm run dev + python server.py`).

8.4 48-Hour Hackathon Execution Timeline

- **Hours 0–12 (Foundation & Setup):** Monorepo initialization, environment setup, base repository cloning, vacuum physics refactoring, and Three.js canvas setup.
- **Hours 12–24 (Model Training & 3D Environment):** PyTorch PPO model training over 500k timesteps, NASA DEM heightmap ingestion, and Dropper X GLTF CAD model integration.
- **Hours 24–36 (Full-Stack WebSocket Integration):** FastAPI WebSocket server linking backend physics to frontend WebGL visualizer at 60 Hz, telemetric HUD buildout, and manual pilot controls.
- **Hours 36–48 (Polish, NASA Data Validation, Demo Video & Submission):** Parameter tuning, slope hazard map verification, recording 30-second demonstration video, and final NASA Space Apps portal submission.

9. Official Source & Reference Links

- **NASA Space Apps Challenge 2026 Track:** Lunar Landing Site Navigator / Space Mission Design Game

- **NASA Moon Trek GIS Data Portal:** https://trek.nasa.gov/moon/
- **RL 6-DOF Simulator Reference:** aryamehr2k Rocket Landing Simulator
- **3D Visualizer Reference:** OsamaZerkawi Three.js Rocket Simulation
- **Official Team Monorepo:** https://github.com/Marwane-Haddane/nasa-hack